

Sentinel – Hybrid Detection Layer: Code to Write

Sentinel – Hybrid Detection Layer: Code to Write

- [config/column_maps.py](#)
- [config/constants.py](#)
- [feature_extractor/feature_builder.py](#)
- [feature_extractor/zeek_parser.py](#)
- [model/preprocessor.py](#)
- [model/dataset.py](#)
- [model/et_ssl.py](#)
- [model/losses.py](#)
- [model/incremental.py](#)
- [detector/scorer.py](#)
- [detector/rules.py](#)
- [detector/ensemble.py](#)
- [detector/classifier.py](#)
- [detector/publisher.py](#)
- [detector/model_loader.py](#)
- [detector/detection_service.py](#)
- [Notebook Skeletons](#)
- [tests/test_feature_builder.py](#)
- [tests/test_losses.py](#)
- [tests/test_rules.py](#)

Sentinel – Hybrid Detection Layer: Code to Write

Every file you need to create, with full skeleton code, imports, class signatures, method signatures, and inline comments explaining what each block must do. **Training runs in Google Colab. Inference runs in Docker.**

config/column_maps.py

```

"""
Column name mappings from each dataset's raw CSV headers
to the unified 20-feature names used throughout the pipeline.
"""

# CIC-Darknet2020 CSV column → unified name
DARKNET_MAP = {
    "Flow Duration": "duration",
    "Total Fwd Packets": "orig_pkts",
    "Total Backward Packets": "resp_pkts",
    "Total Length of Fwd Packets": "orig_bytes",
    "Total Length of Bwd Packets": "resp_bytes",
    "Flow Packets/s": "pkt_rate",
    "Fwd Packet Length Mean": "mean_pkt_size",
    "Bwd Packet Length Mean": "mean_resp_pkt_size",
    "Protocol": "proto_raw", # 6=TCP, 17=UDP
    "Destination Port": "dst_port",
    "Label": "label_raw", # "BENIGN" or attack
    name
}

# UNSW-NB15 CSV column → unified name
UNSW_MAP = {
    "dur": "duration",
    "sbytes": "orig_bytes",
    "dbytes": "resp_bytes",
    "spkts": "orig_pkts",
    "dpkts": "resp_pkts",
    "sload": "pkt_rate", # bits/sec (proxy)
    "smean": "mean_pkt_size",
    "dmean": "mean_resp_pkt_size",
    "proto": "proto_raw",
    "dport": "dst_port",
    "state": "conn_state_raw",
    "service": "service_raw",
    "label": "label_raw", # 0=normal, 1=attack
}

# ISCX VPN-nonVPN CSV column → unified name
ISCX_MAP = {
    "duration": "duration",

```

```
        "total_fpackets":    "orig_pkts",
        "total_bpackets":    "resp_pkts",
        "total_fbytes":      "orig_bytes",
        "total_bbytes":      "resp_bytes",
        "rate":              "pkt_rate",
        "mean":              "mean_pkt_size",
        "Protocol":          "proto_raw",
        "class1":            "label_raw", # "VPN" or app category
    }

    # Unified binary label values for each dataset
    LABEL_NORMAL = {
        "darknet": "BENIGN",
        "unsw":    0,
        "iscx":    "BROWSING", # treat VPN browsing as normal baseline
    }
```

config/constants.py

```

"""
    Shared constants used across feature extraction, rules, and detection
    service.
"""

import numpy as np

# The 20 feature names in order – must match build_feature_vector()
output
FEATURE_NAMES = [
    "duration",
    "orig_bytes",
    "resp_bytes",
    "orig_pkts",
    "resp_pkts",
    "bytes_ratio",
    "pkt_rate",
    "mean_pkt_size",
    "mean_resp_pkt_size",
    "log_duration",
    "log_orig_bytes",
    "log_resp_bytes",
    "conn_state_SF",
    "conn_state_S0",
    "conn_state_REJ",
    "conn_state_RST0",
    "conn_state_RSTR",
    "conn_state_OTH",
    "proto_tcp",
    "proto_udp",
]

FEATURE_DIM = len(FEATURE_NAMES) # 20

# Well-known ports – used by the rule engine
COMMON_PORTS = {
    21, 22, 23, 25, 53, 80, 110, 143, 443, 445,
    465, 587, 993, 995, 3306, 3389, 5432, 6379,
    8080, 8443, 8888,
}

# ET-SSL model dimensions

```

```

ENCODER_HIDDEN_1 = 128
ENCODER_HIDDEN_2 = 256
ENCODER_HIDDEN_3 = 128
EMBEDDING_DIM    = 64  # k in the paper: k << d
PROJECTION_DIM   = 32  # projection head output

# Training hyperparameters (paper values)
BATCH_SIZE       = 256
LEARNING_RATE    = 1e-3
LR_DECAY_FACTOR  = 0.95
LR_DECAY_EPOCHS  = 10
NUM_EPOCHS       = 100
TEMPERATURE_TAU  = 0.1  # contrastive loss temperature
GAMMA            = 0.5  # weight for anomaly detection loss
ALPHA_EMA        = 0.95 # incremental centroid decay factor

# Train/val/test split ratios (paper: 70/15/15)
TRAIN_RATIO = 0.70
VAL_RATIO   = 0.15
TEST_RATIO  = 0.15

# Detection
SCORE_NORMALISE_PERCENTILE = 99 # clip scores above this percentile to
1.0

ENSEMBLE_WEIGHTS = {
    "et_ssl": 0.50,
    "random_forest": 0.25,
    "autoencoder": 0.15,
    "rule_engine": 0.10,
}
ENSEMBLE_THRESHOLD = 0.5

```

feature_extractor/feature_builder.py

```

"""
Converts a raw Zeek conn.log record (dict) into the 20-dim feature vector
that the ET-SSL model expects.

Called by:
- notebooks: to build numpy arrays from dataset CSVs
- detection_service: to build tensors from live Kafka messages
"""

import numpy as np
import math
from config.constants import FEATURE_DIM

# Valid connection states for one-hot encoding
CONN_STATES = ["SF", "S0", "REJ", "RST0", "RSTR", "OTH"]

def build_feature_vector(record: dict) -> np.ndarray:
    """
    Build a float32 feature vector of shape (20,) from a conn.log record.

    Args:
        record: dict with keys matching Zeek conn.log field names or
            unified names from column_maps.py

    Returns:
        np.ndarray of shape (FEATURE_DIM,) dtype float32

    Steps:
        1. Extract scalar fields with safe defaults
        2. Derive ratio/rate features
        3. Apply log1p transforms
        4. Build one-hot vectors for conn_state
        5. Build binary protocol flags
        6. Concatenate into final vector
        7. Replace any NaN/Inf with 0
    """
    # --- 1. Raw field extraction ---
    duration      = float(record.get("duration", 0) or 0)
    orig_bytes    = float(record.get("orig_bytes", 0) or 0)
    resp_bytes    = float(record.get("resp_bytes", 0) or 0)

```

```

orig_pkts      = float(record.get("orig_pkts", 0) or 0)
resp_pkts      = float(record.get("resp_pkts", 0) or 0)
proto          = str(record.get("proto", "")).lower()
conn_state     = str(record.get("conn_state", "OTH") or
"OTH").upper()
service        = str(record.get("service", "") or "").lower()
dst_port       = int(record.get("dst_port", 0) or 0)

# --- 2. Derived features ---
bytes_ratio    = orig_bytes / (resp_bytes + 1.0)
pkt_rate       = orig_pkts / (duration + 1e-6)
mean_pkt_size  = orig_bytes / (orig_pkts + 1.0)
mean_resp_pkt_size = resp_bytes / (resp_pkts + 1.0)

# --- 3. Log transforms (handle heavy-tailed distributions) ---
log_duration   = math.loglp(duration)
log_orig_bytes = math.loglp(orig_bytes)
log_resp_bytes = math.loglp(resp_bytes)

# --- 4. Connection state one-hot (6 states) ---
state_key = conn_state if conn_state in CONN_STATES else "OTH"
conn_state_vec = [1.0 if s == state_key else 0.0 for s in
CONN_STATES]

# --- 5. Protocol binary flags ---
proto_tcp = 1.0 if proto == "tcp" else 0.0
proto_udp = 1.0 if proto == "udp" else 0.0

# --- 6. Concatenate ---
vec = np.array([
    duration,
    orig_bytes,
    resp_bytes,
    orig_pkts,
    resp_pkts,
    bytes_ratio,
    pkt_rate,
    mean_pkt_size,
    mean_resp_pkt_size,
    log_duration,
    log_orig_bytes,
    log_resp_bytes,
    *conn_state_vec,      # 6 values
    proto_tcp,
    proto_udp,

```



```
    ], dtype=np.float32)

    assert vec.shape == (FEATURE_DIM,), f"Expected {FEATURE_DIM}
features, got {vec.shape}"

    # --- 7. Sanitise ---
    vec = np.nan_to_num(vec, nan=0.0, posinf=0.0, neginf=0.0)
    return vec

def build_feature_matrix(records: list[dict]) -> np.ndarray:
    """
    Vectorised version: build (N, 20) matrix from a list of records.
    Used in detection_service for batch processing.
    """
    return np.stack([build_feature_vector(r) for r in records], axis=0)
```

feature_extractor/zeek_parser.py

```

"""
Parses Zeek conn.log JSON messages from the Kafka network-features topic.
Normalises field names to match build_feature_vector() expectations.
"""

import json
import logging

logger = logging.getLogger("sentinel.zeek_parser")

def parse_kafka_message(raw_value: bytes) -> dict | None
    """
    Deserialise a Kafka message from the network-features topic.

    Returns:
        dict with unified field names, or None if the message is
malformed.

    The Zeek JSON format uses field names like: uid, ts, id.orig_h,
id.orig_p, id.resp_h, id.resp_p, proto, service, duration,
orig_bytes, etc.

    Map them to flat unified names here.
    """
    try:
        raw = json.loads(raw_value.decode("utf-8"))
    except (json.JSONDecodeError, UnicodeDecodeError):
        logger.warning("Malformed Kafka message, skipping")
        return None

    # Map Zeek's nested id.* fields to flat names
    record = {
        "uid": raw.get("uid", ""),
        "timestamp": raw.get("ts", ""),
        "src_ip": raw.get("id.orig_h", raw.get("src_ip", "")),
        "src_port": raw.get("id.orig_p", raw.get("src_port", 0)),
        "dst_ip": raw.get("id.resp_h", raw.get("dst_ip", "")),
        "dst_port": raw.get("id.resp_p", raw.get("dst_port", 0)),
        "proto": raw.get("proto", ""),
        "service": raw.get("service", ""),
        "duration": raw.get("duration", 0),

```

```
        "orig_bytes": raw.get("orig_bytes", 0),  
        "resp_bytes": raw.get("resp_bytes", 0),  
        "orig_pkts": raw.get("orig_pkts", 0),  
        "resp_pkts": raw.get("resp_pkts", 0),  
        "conn_state": raw.get("conn_state", "OTH"),  
        "ssl_version": raw.get("ssl_version", None),  
        "dns_query": raw.get("dns_query", None),  
    }  
    return record
```

model/preprocessor.py

```

"""
Preprocessing: RobustScaler wrapper and stochastic Augmenter.
The scaler is fit once on the training set and saved to
artifacts/scaler.pkl.
"""

import numpy as np
import joblib
from sklearn.preprocessing import RobustScaler
from config.constants import FEATURE_DIM


class TrafficScaler:
    """
    Thin wrapper around RobustScaler.
    Fit on training data, then used to transform all splits and live
    data.
    """

    def __init__(self):
        self._scaler = RobustScaler()

    def fit(self, X: np.ndarray) -> "TrafficScaler":
        """
        Fit the scaler on training data X of shape (N, FEATURE_DIM).
        Call this ONLY on the training split.
        """
        self._scaler.fit(X)
        return self

    def transform(self, X: np.ndarray) -> np.ndarray:
        """Scale X and return float32 array of same shape."""
        return self._scaler.transform(X).astype(np.float32)

    def fit_transform(self, X: np.ndarray) -> np.ndarray:
        return self.fit(X).transform(X)

    def save(self, path: str):
        """Persist fitted scaler to disk."""
        joblib.dump(self._scaler, path)

```

```

@classmethod
def load(cls, path: str) -> "TrafficScaler":
    """Load a previously fitted scaler from disk."""
    obj = cls()
    obj._scaler = joblib.load(path)
    return obj

class Augmenter:
    """
    Stochastic data augmentation to generate positive pairs for
    self-supervised contrastive learning (Section 5.2 of guide).

    Given a sample  $x_i$ , returns  $(x_i, x_{i\_aug})$  where  $x_{i\_aug}$  is
    a perturbed version of the same flow.
    """

    def __init__(
        self,
        noise_std: float = 0.05,
        dropout_max: int = 3,
        jitter_range: tuple = (0.9, 1.1),
    ):
        self.noise_std = noise_std
        self.dropout_max = dropout_max
        self.jitter_range = jitter_range

    def augment(self, x: np.ndarray) -> np.ndarray:
        """
        Apply 1–2 randomly chosen augmentations to a single feature
        vector.

        Augmentations (paper Section 5.2):
        - gaussian_noise: add  $N(0, noise\_std)$  to all continuous
        features
        - feature_dropout: zero out up to dropout_max random features
        - scale_jitter: multiply continuous features by  $U(jitter\_range)$ 

        Returns augmented copy (original x is not modified).
        """
        x_aug = x.copy()
        n_augs = np.random.randint(1, 3) # apply 1 or 2 augmentations
        choices = np.random.choice(["noise", "dropout", "jitter"],
                                    size=n_augs, replace=False)

```

```
for aug in choices:
    if aug == "noise":
        x_aug = self._gaussian_noise(x_aug)
    elif aug == "dropout":
        x_aug = self._feature_dropout(x_aug)
    elif aug == "jitter":
        x_aug = self._scale_jitter(x_aug)

return x_aug.astype(np.float32)

def augment_batch(self, X: np.ndarray) -> np.ndarray:
    """Apply augment() to every row of X. Returns (N,
    FEATURE_DIM)."""
    return np.stack([self.augment(x) for x in X], axis=0)

def _gaussian_noise(self, x: np.ndarray) -> np.ndarray:
    noise = np.random.normal(0, self.noise_std,
    size=x.shape).astype(np.float32)
    return x + noise

def _feature_dropout(self, x: np.ndarray) -> np.ndarray:
    n = np.random.randint(1, self.dropout_max + 1)
    idx = np.random.choice(len(x), size=n, replace=False)
    x = x.copy()
    x[idx] = 0.0
    return x

def _scale_jitter(self, x: np.ndarray) -> np.ndarray:
    lo, hi = self.jitter_range
    scale = np.random.uniform(lo, hi,
    size=x.shape).astype(np.float32)
    return x * scale
```

model/dataset.py

```

"""
PyTorch Dataset for contrastive pretraining and supervised fine-tuning.
"""

import numpy as np
import torch
from torch.utils.data import Dataset
from model.preprocessor import Augmenter

class TrafficDataset(Dataset):
    """
    Wraps a numpy feature matrix for use with DataLoader.

    In self-supervised mode (labels=None):
        __getitem__ returns (x_original, x_augmented) – a positive pair.

    In supervised mode (labels provided):
        __getitem__ returns (x_original, x_augmented, label).
    """

    def __init__(
        self,
        X: np.ndarray,
        labels: np.ndarray | None = None,
        augmenter: Augmenter | None = None,
    ):
        """
        Args:
            X: Scaled feature matrix, shape (N, FEATURE_DIM)
            labels: Optional binary labels array, shape (N,).
                   0=normal, 1=anomaly.
            augmenter: Augmenter instance. If None, creates a default
                   one.
        """
        self.X = torch.from_numpy(X).float()
        self.labels = torch.from_numpy(labels).long() if labels is not
        None else None
        self.augmenter = augmenter or Augmenter()

    def __len__(self) -> int:

```

```
        return len(self.X)

    def __getitem__(self, idx: int):
        x_orig = self.X[idx]
        x_aug = torch.from_numpy(
            self.augmenter.augment(x_orig.numpy())
        ).float()

        if self.labels is not None:
            return x_orig, x_aug, self.labels[idx]
        return x_orig, x_aug
```


model/et_ssl.py

```

"""
ET-SSL model: Encoder + Projection Head.

Architecture (Section 6 of guide):
Encoder:      Linear(20→128) → BN → ReLU → Dropout
              Linear(128→256) → BN → ReLU → Dropout
              Linear(256→128) → BN → ReLU
              Linear(128→64) ← embedding z_i
Projection Head: Linear(64→64) → ReLU → Linear(64→32) ← h_i
                  (discarded after training)
"""

import torch
import torch.nn as nn
from config.constants import (
    FEATURE_DIM, ENCODER_HIDDEN_1, ENCODER_HIDDEN_2,
    ENCODER_HIDDEN_3, EMBEDDING_DIM, PROJECTION_DIM,
)

class ETSSLEncoder(nn.Module):
    """
    The core encoder f_theta: maps x_i in R^20 to z_i in R^64.
    This module is KEPT after training and used for inference.
    """

    def __init__(self, dropout: float = 0.3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(FEATURE_DIM, ENCODER_HIDDEN_1),
            nn.BatchNorm1d(ENCODER_HIDDEN_1),
            nn.ReLU(),
            nn.Dropout(dropout),

            nn.Linear(ENCODER_HIDDEN_1, ENCODER_HIDDEN_2),
            nn.BatchNorm1d(ENCODER_HIDDEN_2),
            nn.ReLU(),
            nn.Dropout(dropout),

            nn.Linear(ENCODER_HIDDEN_2, ENCODER_HIDDEN_3),
            nn.BatchNorm1d(ENCODER_HIDDEN_3),

```

```

        nn.ReLU(),

        nn.Linear(ENCODER_HIDDEN_3, EMBEDDING_DIM),
        # No activation on final layer – raw embeddings for distance
scoring
    )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Args:
            x: (batch_size, FEATURE_DIM)
        Returns:
            z: (batch_size, EMBEDDING_DIM)
        """
        return self.net(x)

class ProjectionHead(nn.Module):
    """
    Small MLP projection head used ONLY during contrastive training.
    Maps z_i -> h_i for the NT-Xent loss computation.
    Discard this after training; use z_i directly for anomaly scoring.
    """

    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(EMBEDDING_DIM, EMBEDDING_DIM),
            nn.ReLU(),
            nn.Linear(EMBEDDING_DIM, PROJECTION_DIM),
        )

    def forward(self, z: torch.Tensor) -> torch.Tensor:
        """
        Args:
            z: (batch_size, EMBEDDING_DIM)
        Returns:
            h: (batch_size, PROJECTION_DIM)
        """
        return self.net(z)

class ETSSLModel(nn.Module):
    """
    Combined model for training: Encoder + Projection Head.

```

```
Use encoder.forward() alone at inference time.
"""

def __init__(self, dropout: float = 0.3):
    super().__init__()
    self.encoder = ETSSLEncoder(dropout=dropout)
    self.projector = ProjectionHead()

def forward(self, x: torch.Tensor) -> tuple[torch.Tensor,
torch.Tensor]:
    """
    Returns:
        z: embeddings (batch, EMBEDDING_DIM) – for anomaly scoring
        h: projections (batch, PROJECTION_DIM) – for contrastive loss
    """
    z = self.encoder(x)
    h = self.projector(z)
    return z, h

def encode(self, x: torch.Tensor) -> torch.Tensor:
    """Inference-only shortcut: return embeddings z only."""
    return self.encoder(x)
```

model/losses.py

```

"""
    Loss functions for ET-SSL training (Section 6.3-6.5 of guide).

    NTXentLoss:          Normalised temperature-scaled cross-entropy
(SimCLR).
    AnomalyDetectionLoss: Pushes anomalous embeddings away from the normal
centroid.
    ETSSLoss:           Combines both with weighting factor gamma.
"""

import torch
import torch.nn as nn
import torch.nn.functional as F
from config.constants import TEMPERATURE_TAU, GAMMA


class NTXentLoss(nn.Module):
    """
    NT-Xent loss for contrastive self-supervised learning.

    For a batch of N original flows and their N augmented counterparts,
    we form 2N projections. Each (h_i, h_i_aug) is a positive pair;
    all other 2N-2 combinations are negatives.

    
$$L_{\text{contrastive}} = -(1/N) * \sum_i \log[ \frac{\exp(\text{sim}(h_i, h_{i+}) / \tau)}{\sum_{k \neq i} \exp(\text{sim}(h_i, h_k) / \tau)} ]$$

    """

    def __init__(self, temperature: float = TEMPERATURE_TAU):
        super().__init__()
        self.temperature = temperature

    def forward(self, h_orig: torch.Tensor, h_aug: torch.Tensor) ->
torch.Tensor:
    """
    Args:
        h_orig: (N, PROJECTION_DIM) – projections of original views
        h_aug:  (N, PROJECTION_DIM) – projections of augmented views
    """

```

Returns:

Scalar loss tensor.

Steps:

1. L2-normalise both sets of projections
2. Concatenate into $(2N, \text{PROJECTION_DIM})$
3. Compute full $(2N, 2N)$ cosine similarity matrix
4. Divide by temperature
5. Mask out self-similarity on the diagonal
6. Compute cross-entropy against the positive pair index

"""

`N = h_orig.shape[0]`

`device = h_orig.device`

Step 1: L2 normalise

`h_orig = F.normalize(h_orig, dim=1)`

`h_aug = F.normalize(h_aug, dim=1)`

Step 2: Concatenate → $(2N, D)$

`h_all = torch.cat([h_orig, h_aug], dim=0)`

Step 3: Full cosine similarity matrix $(2N, 2N)$

`sim_matrix = torch.mm(h_all, h_all.T) / self.temperature`

Step 4: Remove self-similarity from denominator

`mask = torch.eye(2 * N, dtype=torch.bool, device=device)`

`sim_matrix = sim_matrix.masked_fill(mask, float("-inf"))`

Step 5: Positive pair indices

For sample i in $[0, N)$: its positive is at position $i+N$

For sample i in $[N, 2N)$: its positive is at position $i-N$

`labels = torch.cat([`
 `torch.arange(N, 2 * N, device=device),`
 `torch.arange(0, N, device=device),`
`]) # shape (2N,)`

Step 6: Cross-entropy loss

`loss = F.cross_entropy(sim_matrix, labels)`

`return loss`

`class AnomalyDetectionLoss(nn.Module):`

"""

Pulls anomalous embeddings away from the normal traffic centroid z_0 .

```

L_anomaly = sum_i [ I(A(t_i)) * ||z_i - z0||^2 ]

Used in Phase 4 (fine-tuning) when labels are available.
"""

def forward(
    self,
    z: torch.Tensor,
    labels: torch.Tensor,
    centroid: torch.Tensor,
) -> torch.Tensor:
    """
    Args:
        z: (N, EMBEDDING_DIM) – embeddings
        labels: (N,) – binary labels, 1=anomalous
        centroid: (EMBEDDING_DIM,) – centre of normal embeddings z0

    Returns:
        Scalar loss tensor.
    """
    # Distance from centroid for each embedding
    dist_sq = torch.sum((z - centroid.unsqueeze(0)) ** 2, dim=1) #
    (N,)

    # Apply only on anomalous samples
    anomaly_mask = (labels == 1).float()
    loss = (anomaly_mask * dist_sq).sum()
    return loss

class ETSSLoss(nn.Module):
    """
    Total loss: L_total = L_contrastive + gamma * L_anomaly

    When no labels are available (pretraining), only L_contrastive is
    used.
    """

    def __init__(
        self,
        temperature: float = TEMPERATURE_TAU,
        gamma: float = GAMMA,
    ):
        super().__init__()
        self.contrastive_loss = NTXentLoss(temperature)
        self.anomaly_loss = AnomalyDetectionLoss()

```

```

        self.gamma = gamma

    def forward(
        self,
        h_orig: torch.Tensor,
        h_aug: torch.Tensor,
        z: torch.Tensor | None = None,
        labels: torch.Tensor | None = None,
        centroid: torch.Tensor | None = None
    ) -> tuple[torch.Tensor, dict]:
        """
        Args:
            h_orig: (N, PROJECTION_DIM)
            h_aug: (N, PROJECTION_DIM)
            z: (N, EMBEDDING_DIM) – required for anomaly loss
            labels: (N,) binary – required for anomaly loss
            centroid: (EMBEDDING_DIM,) – required for anomaly loss

        Returns:
            (total_loss, {"contrastive": float, "anomaly": float})
        """
        l_cont = self.contrastive_loss(h_orig, h_aug)
        info = {"contrastive": l_cont.item(), "anomaly": 0.0}

        if z is not None and labels is not None and centroid is not None:
            l_anom = self.anomaly_loss(z, labels, centroid)
            total = l_cont + self.gamma * l_anom
            info["anomaly"] = l_anom.item()
        else:
            total = l_cont

        return total, info

```

model/incremental.py

```

    """
    Incremental centroid updater for adapting to evolving normal traffic.
    Implements the EMA update from the paper (Section 9 of guide):

        
$$\mu_{\text{norm}}^{(t+1)} = \alpha * \mu_{\text{norm}}^{(t)} + (1-\alpha) * \text{mean}(z_i \text{ for normal } i)$$

    """

    import numpy as np
    import redis
    import json
    import logging
    from config.constants import EMBEDDING_DIM, ALPHA_EMA

    logger = logging.getLogger("sentinel.incremental")
    REDIS_KEY = "et_ssl:normal_centroid"

    class CentroidUpdater:
        """
        Maintains and updates the normal traffic centroid  $\mu_{\text{norm}}$ .
        Persists to Redis so the detection service picks up changes without
        restart.
        """

        def __init__(
            self,
            redis_client: redis.Redis,
            alpha: float = ALPHA_EMA,
        ):
            self.redis = redis_client
            self.alpha = alpha
            self._mu = None  # in-memory cache

        def load(self, path: str):
            """Load initial centroid from numpy file (saved after Phase 2
            training)."""
            self._mu = np.load(path).astype(np.float32)
            self._write_redis()
            logger.info(f"Centroid loaded from {path}")

```



```

def get(self) -> np.ndarray:
    """Return current centroid, fetching from Redis if not cached."""
    if self._mu is None:
        raw = self.redis.get(REDIS_KEY)
        if raw:
            self._mu = np.array(json.loads(raw), dtype=np.float32)
        else:
            raise RuntimeError("No centroid in Redis – run load()
first")

    return self._mu

def update(self, new_normal_embeddings: np.ndarray):
    """
    Apply EMA update with a batch of new normal embeddings.

    Args:
        new_normal_embeddings: (M, EMBEDDING_DIM) – embeddings of
flows
                                                                    scored below the anomaly threshold in
the
                                                                    current window.

    """
    if len(new_normal_embeddings) == 0:
        return
    new_mean = new_normal_embeddings.mean(axis=0).astype(np.float32)
    self._mu = self.alpha * self.get() + (1 - self.alpha) * new_mean
    self._write_redis()
    logger.debug("Centroid updated via EMA")

def _write_redis(self):
    """Persist current centroid to Redis as JSON list."""
    self.redis.set(REDIS_KEY, json.dumps(self._mu.tolist()))

```

detector/scorer.py

```

"""
Vectorised anomaly scoring.
Computes  $S(t_i) = ||z_i - \mu_{\text{norm}}||^2$  for a batch of embeddings.
"""

import numpy as np
import torch
from config.constants import SCORE_NORMALISE_PERCENTILE


def compute_anomaly_scores(
    Z: np.ndarray,
    mu_norm: np.ndarray,
) -> np.ndarray:
    """
    Compute Euclidean distance squared from each embedding to the normal
    centroid.

    Args:
        Z: (N, EMBEDDING_DIM) – encoder output embeddings
        mu_norm: (EMBEDDING_DIM,) – normal traffic centroid

    Returns:
        scores: (N,) float32 – raw anomaly scores (higher = more
    anomalous)
    """
    diff = Z - mu_norm[np.newaxis, :] # (N, D)
    scores = np.sum(diff ** 2, axis=1) # (N,)
    return scores.astype(np.float32)


def normalise_scores(
    scores: np.ndarray,
    p99: float | None = None,
) -> np.ndarray:
    """
    Normalise raw distance scores to [0, 1] range by clipping at the
    SCORE_NORMALISE_PERCENTILE-th percentile of the current batch.

    Args:
        scores: (N,) raw scores
    """

```

p99: Pre-computed clip value. If None, computed from current batch.

Returns:

(N,) float32 scores in [0, 1]

"""

```
clip_val = p99 if p99 is not None else np.percentile(scores,  
SCORE_NORMALISE_PERCENTILE)
```

```
clipped = np.clip(scores, 0, clip_val)
```

```
return (clipped / (clip_val + 1e-8)).astype(np.float32)
```

detector/rules.py

```

"""
Deterministic rule engine.
Produces a vote in [0, 1] for each flow based on fast heuristics.
Runs in parallel with model scoring – no model loading required.
"""

from config.constants import COMMON_PORTS


class RuleEngine:
    """
    Each check_* method returns a float vote in [0, 1].
    0 = definitely normal, 1 = definitely anomalous.
    The rule() method returns the maximum vote across all checks.
    """

    def vote(self, record: dict) -> tuple[float, str]:
        """
        Score a single flow record against all rules.

        Args:
            record: dict with unified field names (same as
                build_feature_vector input)

        Returns:
            (vote, rule_name): highest vote and which rule triggered it.
            Returns (0.0, "none") if no rule matches.
        """
        checks = [
            ("port_scan", self._port_scan(record)),
            ("dns_exfiltration", self._dns_exfil(record)),
            ("large_upload", self._large_upload(record)),
            ("short_burst", self._short_burst(record)),
            ("unusual_port", self._unusual_port(record)),
            ("tls_nonstandard", self._tls_nonstandard(record)),
        ]
        best_rule, best_vote = "none", 0.0
        for rule_name, v in checks:
            if v > best_vote:
                best_vote = v
                best_rule = rule_name

```

```

        return best_vote, best_rule

def _port_scan(self, r: dict) -> float:
    """SYN-only packets to single port with zero response – port
    scan."""
    if (r.get("orig_pkts", 0) > 100
        and r.get("resp_bytes", 0) == 0
        and r.get("conn_state", "") == "S0"):
        return 1.0
    return 0.0

def _dns_exfil(self, r: dict) -> float:
    """Large DNS query – potential DNS tunnelling."""
    if r.get("dst_port", 0) == 53 and r.get("orig_bytes", 0) > 500:
        return 0.9
    return 0.0

def _large_upload(self, r: dict) -> float:
    """Data exfiltration: much more sent than received over long
    flow."""
    orig = r.get("orig_bytes", 0) or 0
    resp = r.get("resp_bytes", 0) or 0
    dur = r.get("duration", 0) or 0
    if orig > resp * 50 and dur > 60:
        return 0.7
    return 0.0

def _short_burst(self, r: dict) -> float:
    """Very short flow with many packets – possible scanning or C2
    beacon."""
    dur = r.get("duration", 0) or 0
    pkts = r.get("orig_pkts", 0) or 0
    if dur < 0.5 and pkts > 20:
        return 0.8
    return 0.0

def _unusual_port(self, r: dict) -> float:
    """TCP connection to a non-standard port."""
    proto = str(r.get("proto", "")).lower()
    port = r.get("dst_port", 0) or 0
    if proto == "tcp" and port not in COMMON_PORTS and port > 1024:
        return 0.3
    return 0.0

def _tls_nonstandard(self, r: dict) -> float:

```

```
    """TLS on a port that doesn't normally carry it."""  
    ssl = r.get("ssl_version", None)  
    port = r.get("dst_port", 0 or 0)  
    if ssl and port not in (443, 8443, 993, 465, 587, 636, 5061):  
        return 0.4  
    return 0.0
```

detector/ensemble.py

```

"""
Weighted ensemble that combines ET-SSL, Random Forest,
Autoencoder, and Rule Engine votes into a final anomaly score.
"""

import numpy as np
from config.constants import ENSEMBLE_WEIGHTS, ENSEMBLE_THRESHOLD

def ensemble_vote(
    et_ssl_score: float,
    rf_prob: float,
    ae_score: float,
    rule_vote: float,
) -> tuple[float, bool]:
    """
    Combine model outputs into a single weighted anomaly score.

    Args:
        et_ssl_score: Normalised ET-SSL score in [0, 1]
        rf_prob: Random Forest P(anomaly) in [0, 1]
        ae_score: Normalised autoencoder reconstruction error in [0,
1]

        rule_vote: Rule engine vote in [0, 1]

    Returns:
        (weighted_score, is_anomaly):
            weighted_score: float in [0, 1]
            is_anomaly: bool, True if weighted_score >
ENSEMBLE_THRESHOLD
    """
    w = ENSEMBLE_WEIGHTS
    score = (
        w["et_ssl"] * et_ssl_score +
        w["random_forest"] * rf_prob +
        w["autoencoder"] * ae_score +
        w["rule_engine"] * rule_vote
    )
    return float(score), score > ENSEMBLE_THRESHOLD

```

```
def ensemble_batch(
    et_ssl_scores: np.ndarray,
    rf_probs:      np.ndarray,
    ae_scores:     np.ndarray,
    rule_votes:    np.ndarray,
) -> tuple[np.ndarray, np.ndarray]:
    """
    Vectorised version for processing a full batch simultaneously.
    All inputs are shape (N,). Returns (scores, flags) both shape (N,).
    """
    w = ENSEMBLE_WEIGHTS
    scores = (
        w["et_ssl"] * et_ssl_scores +
        w["random_forest"] * rf_probs +
        w["autoencoder"] * ae_scores +
        w["rule_engine"] * rule_votes
    )
    flags = scores > ENSEMBLE_THRESHOLD
    return scores.astype(np.float32), flags
```


detector/classifier.py

```

    """
    Maps ensemble vote + rule signal to a human-readable classification
    string.
    """

def classify(
    weighted_score: float,
    rule_name: str,
    duration: float,
    service: str,
) -> str:
    """
    Determine attack classification from the dominant signal.

    Classification priority:
    1. If a specific rule fired, use its classification
    2. Otherwise classify by ET-SSL score + flow characteristics

    Returns one of:
    port_scan | dns_tunneling | data_exfiltration | exploit_attempt |
    c2_communication | encrypted_anomaly | traffic_anomaly
    """
    # Rule-based classifications take priority
    rule_map = {
        "port_scan": "port_scan",
        "dns_exfiltration": "dns_tunneling",
        "large_upload": "data_exfiltration",
    }
    if rule_name in rule_map:
        return rule_map[rule_name]

    # Score-based heuristics
    if weighted_score >= 0.8:
        if duration < 2.0:
            return "exploit_attempt"
        else:
            return "c2_communication"
    elif weighted_score >= 0.5:
        if service in ("", "-", "unknown"):
            return "encrypted_anomaly"

```

```
        return "traffic_anomaly"  
  
    return "traffic_anomaly"
```

detector/publisher.py

```

"""
Publishes AnomalyAlert messages to the Kafka anomaly-alerts topic.
"""

import json
import uuid
import logging
from datetime import datetime, timezone
from confluent_kafka import Producer

logger = logging.getLogger("sentinel.publisher")
TOPIC = "anomaly-alerts"

class AlertPublisher:
    """
    Wraps the Kafka producer to publish structured AnomalyAlert JSON
messages.
    """

    def __init__(self, bootstrap_servers: str):
        self._producer = Producer({"bootstrap.servers":
bootstrap_servers})

    def publish(
        self,
        record: dict,
        classification: str,
        weighted_score: float,
        model_votes: list[dict],
    ):
        """
        Build and publish one AnomalyAlert to the anomaly-alerts topic.

        Args:
            record:          Original Zeek conn.log record dict
            classification:  Human-readable attack type string
            weighted_score:  Final ensemble score in [0, 1]
            model_votes:     List of per-model vote dicts:
                           [{"model_name": str, "label": str,
                               "score": float, "confidence": float}, ...]

```

```

"""
alert = {
    "alert_id":      f"alert-{uuid.uuid4().hex[:12]}",
    "timestamp":     datetime.now(timezone.utc).isoformat(),
    "classification": classification,
    "anomaly_score": round(float(weighted_score), 4),
    "model_votes":   model_votes,
    "feature_vector": {
        "src_ip":      record.get("src_ip", ""),
        "dst_ip":      record.get("dst_ip", ""),
        "src_port":    record.get("src_port", 0),
        "dst_port":    record.get("dst_port", 0),
        "proto":       record.get("proto", ""),
        "duration":    record.get("duration", 0),
        "orig_bytes":  record.get("orig_bytes", 0),
        "resp_bytes":  record.get("resp_bytes", 0),
        "orig_pkts":   record.get("orig_pkts", 0),
        "resp_pkts":   record.get("resp_pkts", 0),
        "conn_state":  record.get("conn_state", ""),
        "service":     record.get("service", ""),
        "ssl_version": record.get("ssl_version"),
        "dns_query":   record.get("dns_query"),
        "bytes_ratio": round(record.get("orig_bytes", 0) /
                              (record.get("resp_bytes", 1) + 1), 4),
        "pkt_rate":    round(record.get("orig_pkts", 0) /
                              (record.get("duration", 1) + 1e-6), 4),
        "is_dns":      1 if record.get("dst_port") == 53 else 0,
    },
}

self._producer.produce(
    TOPIC,
    key=alert["alert_id"].encode(),
    value=json.dumps(alert).encode(),
    callback=self._delivery_report,
)

self._producer.poll(0)

def flush(self):
    self._producer.flush()

@staticmethod
def _delivery_report(err, msg):
    if err:
        logger.error(f"Alert delivery failed: {err}")

```


detector/model_loader.py

```

"""
Loads all trained artifacts from disk at startup.
Returns a dict of ready-to-use objects.
"""

import torch
import numpy as np
import joblib
import json
import logging
from pathlib import Path
from model.et_ssl import ETSSLEncoder
from model.preprocessor import TrafficScaler

logger = logging.getLogger("sentinel.model_loader")
ARTIFACTS_DIR = Path("artifacts")

def load_all(device: str = "cpu") -> dict:
    """
    Load all model artifacts needed for inference.

    Returns dict with keys:
        encoder: ETSSLEncoder in eval mode on `device`
        scaler: fitted TrafficScaler
        centroid: np.ndarray (EMBEDDING_DIM,)
        threshold: float delta*
        rf_model: fitted sklearn RandomForestClassifier
        ae_model: trained autoencoder nn.Module in eval mode

    Raises FileNotFoundError if any required artifact is missing.
    """
    artifacts = {}

    # ET-SSL encoder
    encoder = ETSSLEncoder()
    state = torch.load(ARTIFACTS_DIR / "encoder.pt",
map_location=device)
    encoder.load_state_dict(state)
    encoder.eval()
    encoder.to(device)

```

```

artifacts["encoder"] = encoder
logger.info("ET-SSL encoder loaded")

# Scaler
artifacts["scaler"] = TrafficScaler.load(ARTIFACTS_DIR /
"scaler.pkl")
logger.info("RobustScaler loaded")

# Normal centroid
artifacts["centroid"] = np.load(ARTIFACTS_DIR /
"normal_centroid.npy")
logger.info("Normal centroid loaded")

# Decision threshold
with open(ARTIFACTS_DIR / "threshold.json") as f:
    artifacts["threshold"] = json.load(f)["delta"]
logger.info(f"Threshold loaded: delta={artifacts['threshold']:.4f}")

# Random Forest
artifacts["rf_model"] = joblib.load(ARTIFACTS_DIR / "rf_model.pkl")
logger.info("Random Forest loaded")

# Autoencoder (optional – skip if file missing)
ae_path = ARTIFACTS_DIR / "autoencoder.pt"
if ae_path.exists():
    from model.et_ssl import ETSSLEncoder as AEEncoder
    # Autoencoder reuses a similar architecture – see notebook 02 for
full def
    artifacts["ae_model"] = _load_autoencoder(ae_path, device)
    logger.info("Autoencoder loaded")
else:
    artifacts["ae_model"] = None
    logger.warning("autoencoder.pt not found – AE vote will be 0")

return artifacts

def _load_autoencoder(path: Path, device: str):
    """Internal: load autoencoder state dict."""
    import torch.nn as nn
    from config.constants import FEATURE_DIM, ENCODER_HIDDEN_1,
EMBEDDING_DIM

    # Simple symmetric autoencoder – must match architecture used in
training

```

```
ae = nn.Sequential(  
    nn.Linear(FEATURE_DIM, ENCODER_HIDDEN_1), nn.ReLU(),  
    nn.Linear(ENCODER_HIDDEN_1, EMBEDDING_DIM), nn.ReLU(),  
    nn.Linear(EMBEDDING_DIM, ENCODER_HIDDEN_1), nn.ReLU(),  
    nn.Linear(ENCODER_HIDDEN_1, FEATURE_DIM),  
)  
ae.load_state_dict(torch.load(path, map_location=device))  
ae.eval()  
ae.to(device)  
return ae
```

detector/detection_service.py

```

    """
    Main detection service loop.
    Consumes network-features from Kafka, scores each batch, publishes
    alerts.
    """

    import time
    import logging
    import numpy as np
    import torch
    import json
    from confluent_kafka import Consumer, KafkaError

    from feature_extractor.zeek_parser import parse_kafka_message
    from feature_extractor.feature_builder import build_feature_matrix
    from detector.scorer import compute_anomaly_scores, normalise_scores
    from detector.ensemble import ensemble_batch
    from detector.classifier import classify
    from detector.rules import RuleEngine
    from detector.publisher import AlertPublisher
    from detector.model_loader import load_all
    from model.incremental import CentroidUpdater
    import redis

    logger = logging.getLogger("sentinel.detector")

    KAFKA_BOOTSTRAP = "localhost:9092"
    INPUT_TOPIC = "network-features"
    GROUP_ID = "hybrid-detector"
    BATCH_SIZE = 256
    CENTROID_REFRESH = 60 # seconds between Redis centroid refreshes
    INCREMENTAL_EVERY = 10_000 # update centroid every N flows

    class DetectionService:
        """
        Main service that ties together all detection components.

        Lifecycle:
        1. __init__: load all artifacts
        2. start(): enter Kafka poll loop

```

```

3. _process_batch(): score + publish for one batch
4. Graceful shutdown on KeyboardInterrupt/SIGTERM
"""

def __init__(self):
    logger.info("Loading model artifacts...")
    self.device = "cuda" if torch.cuda.is_available() else "cpu"
    self.artifacts = load_all(device=self.device)
    self.rules = RuleEngine()
    self.publisher = AlertPublisher(KAFKA_BOOTSTRAP)
    self.redis_cl = redis.Redis(host="localhost", port=6379)

    decode_responses=True)

    self.updater = CentroidUpdater(self.redis_cl)
    self.updater.load("artifacts/normal_centroid.npy")

    # Kafka consumer
    self.consumer = Consumer({
        "bootstrap.servers": KAFKA_BOOTSTRAP,
        "group.id": GROUP_ID,
        "auto.offset.reset": "latest",
        "enable.auto.commit": False
    })

    self.consumer.subscribe([INPUT_TOPIC])

    self._flow_count = 0
    self._last_refresh = time.time()
    self._running = False
    logger.info(f"DetectionService ready on device={self.device}")

def start(self):
    """Enter the main processing loop."""
    self._running = True
    logger.info("Polling Kafka network-features...")

    buffer = [] # accumulate records until batch is full
    raw_recs = [] # keep original records for alert publishing

    try:
        while self._running:
            msg = self.consumer.poll(timeout=0.5)

            if msg is None:
                # Flush partial batch after idle
                if buffer:
                    self._process_batch(buffer, raw_recs)

```

```

        buffer, raw_recs = [], []

        continue

    if msg.error():
        if msg.error().code() != KafkaError._PARTITION_EOF:
            logger.error(f"Kafka error: {msg.error()}")
            continue

    record = parse_kafka_message(msg.value())
    if record is None:
        continue

    buffer.append(record)
    raw_recs.append(record)

    if len(buffer) >= BATCH_SIZE:
        self._process_batch(buffer, raw_recs)
        self.consumer.commit(asynchronous=True)
        buffer, raw_recs = [], []

    # Refresh centroid from Redis periodically
    if time.time() - self._last_refresh > CENTROID_REFRESH:
        self.artifacts["centroid"] = self.updater.get()
        self._last_refresh = time.time()

except KeyboardInterrupt:
    logger.info("Shutdown signal received")
finally:
    self.publisher.flush()
    self.consumer.close()
    logger.info("DetectionService stopped")

def _process_batch(self, records: list[dict], raw: list[dict]):
    """
    Score one batch of flows and publish alerts for anomalies.

    Steps:
    1. Build feature matrix X (N, 20)
    2. Scale X with RobustScaler
    3. Encode with ET-SSL encoder -> embeddings Z (N, 64)
    4. Compute anomaly scores  $S_i = ||z_i - \mu_{norm}||^2$ 
    5. Normalise scores to [0,1]
    6. Random Forest predict_proba -> rf_probs (N,)
    7. Autoencoder reconstruction error -> ae_scores (N,)
    8. Rule engine vote for each flow -> rule_votes (N,)
    """

```

```

        9. Ensemble weighted vote -> final_scores, anomaly_flags (N,)
        10. For each flagged flow: classify and publish AnomalyAlert
        11. Update incremental centroid with normal embeddings
    """
    t0 = time.time()
    N = len(records)

    # Step 1-2: Feature matrix + scaling
    X = build_feature_matrix(records) # (N, 20)

    float32
    X_sc = self.artifacts["scaler"].transform(X) # (N, 20)

    scaled

    # Step 3: Encode
    with torch.no_grad():
        X_t = torch.from_numpy(X_sc).to(self.device)
        Z = self.artifacts["encoder"](X_t).cpu().numpy() # (N, 64)

    # Step 4-5: ET-SSL scores
    centroid = self.artifacts["centroid"]
    raw_scores = compute_anomaly_scores(Z, centroid)
    et_scores = normalise_scores(raw_scores)

    # Step 6: Random Forest
    rf_probs = self.artifacts["rf_model"].predict_proba(X_sc)[:, 1]

    # P(anomaly)

    # Step 7: Autoencoder
    ae_scores = self._autoencoder_scores(X_sc)

    # Step 8: Rules
    rule_votes = np.array([self.rules.vote(r)[0] for r in records],
dtype=np.float32)
    rule_names = [self.rules.vote(r)[1] for r in records]

    # Step 9: Ensemble
    final_scores, anomaly_flags = ensemble_batch(et_scores, rf_probs,
ae_scores, rule_votes)

    # Step 10: Publish alerts
    n_alerts = 0
    for i in range(N):
        if anomaly_flags[i]:
            classification = classify(
                final_scores[i],

```

```

        rule_names[i],
        records[i].get("duration", 0),
        records[i].get("service", ""),
    )
    model_votes = [
        {"model_name": "et_ssl", "label": "anomaly",
         "score": round(float(et_scores[i]), 4), "confidence": round(float(et_scores[i]),
         4)},
        {"model_name": "random_forest", "label": "anomaly",
         "score": round(float(rf_probs[i]), 4), "confidence": round(float(rf_probs[i]),
         4)},
        {"model_name": "autoencoder", "label": "anomaly"
         if ae_scores[i] > 0.5 else "normal", "score": round(float(ae_scores[i]), 4),
         "confidence": round(float(ae_scores[i]), 4)},
        {"model_name": "rule_engine", "label": "anomaly"
         if rule_votes[i] > 0 else "normal", "score": round(float(rule_votes[i]), 4),
         "confidence": round(float(rule_votes[i]), 4)},
    ]
    self.publisher.publish(raw[i], classification,
                           final_scores[i], model_votes)
    n_alerts += 1

    # Step 11: Incremental centroid update using normal embeddings
    self._flow_count += N
    if self._flow_count % INCREMENTAL_EVERY < N:
        normal_mask = ~anomaly_flags
        if normal_mask.any():
            self.updater.update(Z[normal_mask])

    elapsed_ms = (time.time() - t0) * 1000
    logger.info(f"Batch: {N} flows, {n_alerts} alerts,
                {elapsed_ms:.1f}ms ({elapsed_ms/!:.2f}ms/flow)")

```

Notebook Skeletons

notebooks/01_data_preparation.ipynb — Key cells

```

# Cell 1: Install + mount
from google.colab import drive
drive.mount('/content/drive')
!pip install kaggle pandas pyarrow scikit-learn numpy

# Cell 2: Download datasets
import os
os.environ["KAGGLE_USERNAME"] = "YOUR_USERNAME"
os.environ["KAGGLE_KEY"]      = "YOUR_API_KEY"
!kaggle datasets download -d dhoogla/cicdarknet2020 -p
/content/data/darknet --unzip
!kaggle datasets download -d dhoogla/unswnb15 -p /content/data/unswnb15
--unzip

# Cell 3: Load + map columns
import pandas as pd
from config.column_maps import DARKNET_MAP, UNSW_MAP

df_dark = pd.read_csv("/content/data/darknet/Darknet.csv")
df_dark = df_dark.rename(columns=DARKNET_MAP)
df_dark["label"] = (df_dark["label_raw"] != "BENIGN").astype(int)

df_unsw = pd.read_csv("/content/data/unswnb15/UNSW_NB15_training-set.csv")
df_unsw = df_unsw.rename(columns=UNSW_MAP)
df_unsw["label"] = df_unsw["label_raw"].astype(int)

# Cell 4: Clean
def clean(df, name):
    before = len(df)
    df = df.dropna(subset=[
        "duration", "orig_bytes", "resp_bytes", "orig_pkts", "resp_pkts"])
    df = df[df["duration"] > 0]
    df = df.drop_duplicates()
    import numpy as np
    for col in
        ["orig_bytes", "resp_bytes", "orig_pkts", "resp_pkts", "duration"]:
        if col in df.columns:
            df = df[np.isfinite(df[col])]

```

```

        print(f"{name}: {before} -> {len(df)} rows, anomaly rate:
{df['label'].mean():.3f}")
    return df

df_dark = clean(df_dark, "CIC-Darknet2020")
df_unsw = clean(df_unsw, "UNSW-NB15")

# Cell 5: Build feature matrices
import numpy as np, sys
sys.path.insert(0, '/content/Sentinel/hybrid-detection')
from feature_extractor.feature_builder import build_feature_vector

def df_to_features(df):
    records = df.to_dict("records")
    X = np.stack([build_feature_vector(r) for r in records])
    y = df["label"].values.astype(np.int32)
    return X, y

X_dark, y_dark = df_to_features(df_dark)
X_unsw, y_unsw = df_to_features(df_unsw)
print("Darknet features:", X_dark.shape)
print("UNSW features:", X_unsw.shape)

# Cell 6: Train/val/test split
from sklearn.model_selection import train_test_split

def split(X, y, name):
    X_tr, X_tmp, y_tr, y_tmp = train_test_split(X, y, test_size=0.30,
stratify=y, random_state=42)
    X_v, X_te, y_v, y_te = train_test_split(X_tmp, y_tmp,
test_size=0.50, stratify=y_tmp, random_state=42)
    print(f"{name}: train={len(X_tr)} val={len(X_v)} test={len(X_te)}")
    return X_tr, X_v, X_te, y_tr, y_v, y_te

X_tr_d, X_v_d, X_te_d, y_tr_d, y_v_d, y_te_d = split(X_dark, y_dark,
"Darknet")
X_tr_u, X_v_u, X_te_u, y_tr_u, y_v_u, y_te_u = split(X_unsw, y_unsw,
"UNSW")

# Cell 7: Fit scaler on training set ONLY
from model.preprocessor import TrafficScaler
import joblib

scaler = TrafficScaler()
X_tr_d_sc = scaler.fit_transform(X_tr_d) # fit here

```

```
X_v_d_sc = scaler.transform(X_v_d)
X_te_d_sc = scaler.transform(X_te_d)

DRIVE = "/content/drive/MyDrive/sentinel_artifacts/"
import os; os.makedirs(DRIVE, exist_ok=True)
scaler.save(f"{DRIVE}/scaler.pkl")
print("Scaler saved")

# Cell 8: Save parquet splits
import pyarrow as pa, pyarrow.parquet as pq

def save_split(X_sc, y, name, split_name):
    import pandas as pd
    df = pd.DataFrame(X_sc)
    df["label"] = y
    path = f"{DRIVE}/{name}_{split_name}.parquet"
    df.to_parquet(path, index=False)
    print(f"Saved {path}")

save_split(X_tr_d_sc, y_tr_d, "darknet", "train")
save_split(X_v_d_sc, y_v_d, "darknet", "val")
save_split(X_te_d_sc, y_te_d, "darknet", "test")
```


notebooks/02_pretrain_etssl.ipynb — Key cells

```

# Cell 1: Setup
import torch, numpy as np
from torch.utils.data import DataLoader
import pandas as pd, sys
sys.path.insert(0, '/content/Sentinel/hybrid-detection')

from model.et_ssl import ETSSLModel
from model.losses import ETSSLLoss
from model.dataset import TrafficDataset
from config.constants import BATCH_SIZE, LEARNING_RATE, NUM_EPOCHS,
LR_DECAY_FACTOR, LR_DECAY_EPOCHS

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
DRIVE = "/content/drive/MyDrive/sentinel_artifacts/"

# Cell 2: Load training data
df_train = pd.read_parquet(f"{DRIVE}/darknet_train.parquet")
X_train = df_train.drop(columns=["label"]).values.astype(np.float32)
# Labels NOT used in Phase 1 (self-supervised)
dataset = TrafficDataset(X_train, labels=None)
loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True,
                    num_workers=2, pin_memory=True, drop_last=True)
print(f"Training samples: {len(dataset)}, batches: {len(loader)}")

# Cell 3: Instantiate model and optimizer
model = ETSSLModel().to(device)
criterion = ETSSLLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
scheduler = torch.optim.lr_scheduler.StepLR(optimizer,
step_size=LR_DECAY_EPOCHS,
                                gamma=LR_DECAY_FACTOR)

# Cell 4: Training loop
for epoch in range(1, NUM_EPOCHS + 1):
    model.train()
    epoch_loss = 0.0

    for batch in loader:
        x_orig, x_aug = batch[0].to(device), batch[1].to(device)

        # Forward pass: get projections for both views
        z_orig, h_orig = model(x_orig)

```

```
z_aug, h_aug = model(x_aug)

# Compute NT-Xent loss (no labels in Phase 1)
loss, info = criterion(h_orig, h_aug)

optimizer.zero_grad()
loss.backward()
optimizer.step()

epoch_loss += loss.item()

scheduler.step()
avg_loss = epoch_loss / len(loader)
print(f"Epoch {epoch:3d}/{NUM_EPOCHS} | loss={avg_loss:.4f} | lr={scheduler.get_last_lr()[0]:.6f}")

# Save checkpoint every 10 epochs
if epoch % 10 == 0:
    torch.save(model.encoder.state_dict(), f"{DRIVE}/encoder_epoch{epoch}.pt")

# Cell 5: Save final encoder
torch.save(model.encoder.state_dict(), f"{DRIVE}/encoder.pt")
print("Encoder saved to Drive")
```

notebooks/03_threshold_calibration.ipynb — Key cells

```

# Cell 1: Load encoder + val set
import torch, numpy as np, json
import pandas as pd, sys
sys.path.insert(0, '/content/Sentinel/hybrid-detection')

from model.et_ssl import ETSSLEncoder
from detector.scorer import compute_anomaly_scores
from sklearn.metrics import f1_score

DRIVE = "/content/drive/MyDrive/sentinel_artifacts/"
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

encoder = ETSSLEncoder().to(device)
encoder.load_state_dict(torch.load(f"{DRIVE}/encoder.pt",
map_location=device))
encoder.eval()

df_val = pd.read_parquet(f"{DRIVE}/darknet_val.parquet")
X_val = df_val.drop(columns=["label"]).values.astype(np.float32)
y_val = df_val["label"].values

# Cell 2: Compute normal centroid from normal training samples
df_train = pd.read_parquet(f"{DRIVE}/darknet_train.parquet")
X_tr = df_train.drop(columns=["label"]).values.astype(np.float32)
y_tr = df_train["label"].values
X_normal = X_tr[y_tr == 0]

with torch.no_grad():
    Z_normal =
encoder(torch.from_numpy(X_normal).to(device)).cpu().numpy()

mu_norm = Z_normal.mean(axis=0)
np.save(f"{DRIVE}/normal_centroid.npy", mu_norm)
print(f"Centroid shape: {mu_norm.shape}, saved.")

# Cell 3: Score validation set
with torch.no_grad():
    Z_val = encoder(torch.from_numpy(X_val).to(device)).cpu().numpy()

raw_scores = compute_anomaly_scores(Z_val, mu_norm)

# Cell 4: Find F1-optimal threshold

```

```
thresholds = np.linspace(raw_scores.min(), raw_scores.max(), 200)
best_f1, best_delta = 0.0, 0.0

for delta in thresholds:
    preds = (raw_scores > delta).astype(int)
    f1     = f1_score(y_val, preds, zero_division=0)
    if f1 > best_f1:
        best_f1     = f1
        best_delta = delta

print(f"Best delta={best_delta:.4f}, F1={best_f1:.4f}")
with open(f"{DRIVE}/threshold.json", "w") as f:
    json.dump({"delta": float(best_delta), "val_f1": float(best_f1)}, f)
```

notebooks/05_evaluate.ipynb — Key cells

```

# Cell: Compute all metrics
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, roc_auc_score, roc_curve, precision_recall_curve,
)

def evaluate(y_true, y_pred, y_score, name):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    metrics = {
        "dataset": name,
        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(y_true, y_pred, zero_division=0),
        "recall": recall_score(y_true, y_pred, zero_division=0),
        "f1": f1_score(y_true, y_pred, zero_division=0),
        "fpr": fp / (fp + tn + 1e-8),
        "auc": roc_auc_score(y_true, y_score),
    }
    for k, v in metrics.items():
        if k != "dataset":
            print(f" {k:12s}: {v:.4f}")
    return metrics

# Baseline: Random Forest
from sklearn.ensemble import RandomForestClassifier
import joblib
rf = RandomForestClassifier(n_estimators=100, n_jobs=-1, random_state=42)
rf.fit(X_train, y_train) # X_train, y_train from Cell 1
y_rf_prob = rf.predict_proba(X_test)[: , 1]
y_rf_pred = (y_rf_prob > 0.5).astype(int)
joblib.dump(rf, f"{DRIVE}/rf_model.pkl")
print("Random Forest:"); evaluate(y_test, y_rf_pred, y_rf_prob, "RF")

# ET-SSL
print("ET-SSL:"); evaluate(y_test, y_et_pred, y_et_score_norm,
"ET-SSL")

# UMAP visualisation
import umap
reducer = umap.UMAP(n_components=2, random_state=42)
Z_2d = reducer.fit_transform(Z_test)
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 7))

```

```
plt.scatter(Z_2d[y_test==0, 0], Z_2d[y_test==0, 1], s=2, alpha=0.3,  
label="Normal")  
plt.scatter(Z_2d[y_test==1, 0], Z_2d[y_test==1, 1], s=2, alpha=0.5,  
c="red", label="Anomaly")  
plt.legend(); plt.title("ET-SSL Embedding Space (UMAP)")  
plt.savefig(f"{DRIVE}/umap_embeddings.png", dpi=150)  
plt.show()
```

tests/test_feature_builder.py

```

"""Unit tests for feature_builder.py"""
import numpy as np
import pytest
from feature_extractor.feature_builder import build_feature_vector
from config.constants import FEATURE_DIM


def make_record(**kwargs):
    defaults = {
        "duration": 1.5, "orig_bytes": 1024, "resp_bytes": 512,
        "orig_pkts": 10, "resp_pkts": 8, "proto": "tcp",
        "conn_state": "SF", "service": "ssl", "dst_port": 443
    }
    return {**defaults, **kwargs}


def test_output_shape():
    vec = build_feature_vector(make_record())
    assert vec.shape == (FEATURE_DIM,)


def test_dtype():
    vec = build_feature_vector(make_record())
    assert vec.dtype == np.float32


def test_no_nan():
    vec = build_feature_vector(make_record(orig_bytes=0, resp_bytes=0,
    duration=0))
    assert not np.any(np.isnan(vec))


def test_missing_fields():
    vec = build_feature_vector({}) # all defaults applied
    assert vec.shape == (FEATURE_DIM,)


def test_conn_state_onehot():
    vec_sf = build_feature_vector(make_record(conn_state="SF"))
    vec_s0 = build_feature_vector(make_record(conn_state="S0"))
    # SF is index 0, S0 is index 1 in one-hot
    assert vec_sf[12] == 1.0 and vec_sf[13] == 0.0
    assert vec_s0[12] == 0.0 and vec_s0[13] == 1.0

```

tests/test_losses.py

```
"""Unit tests for NT-Xent loss."""
import torch
from model.losses import NTXentLoss

def test_ntxent_positive_loss():
    """Loss should be > 0 for random embeddings."""
    loss_fn = NTXentLoss(temperature=0.1)
    h_orig = torch.randn(16, 32)
    h_aug = torch.randn(16, 32)
    loss = loss_fn(h_orig, h_aug)
    assert loss.item() > 0

def test_ntxent_perfect_pairs():
    """Loss should be near 0 if original == augmented (perfect positive
pairs)."""
    loss_fn = NTXentLoss(temperature=0.07)
    h = torch.randn(16, 32)
    loss = loss_fn(h, h.clone())
    assert loss.item() < 0.1

def test_ntxent_batch_size_invariant():
    """Loss should be computable for different batch sizes."""
    loss_fn = NTXentLoss()
    for bs in [8, 32, 128]:
        h_o = torch.randn(bs, 32)
        h_a = torch.randn(bs, 32)
        loss = loss_fn(h_o, h_a)
        assert not torch.isnan(loss)
```


tests/test_rules.py

```
"""Unit tests for rule engine."""
from detector.rules import RuleEngine

engine = RuleEngine()

def make_record(**kwargs):
    return {"orig_pkts": 1, "resp_bytes": 100, "orig_bytes": 100,
            "conn_state": "SF", "dst_port": 80, "proto": "tcp",
            "duration": 1.0, "service": "http", "ssl_version": None,
            **kwargs}

def test_port_scan():
    r = make_record(orig_pkts=200, resp_bytes=0, conn_state="S0")
    v, n = engine.vote(r)
    assert n == "port_scan" and v == 1.0

def test_dns_exfil():
    r = make_record(dst_port=53, orig_bytes=1000)
    v, n = engine.vote(r)
    assert n == "dns_exfiltration" and v == 0.9

def test_normal_traffic():
    r = make_record(orig_pkts=5, resp_bytes=200, conn_state="SF",
                    dst_port=443)
    v, n = engine.vote(r)
    assert v == 0.0 and n == "none"
```

Code guide version 1.0 | Sentinel Hybrid Detection Layer | ET-SSL (Sattar et al. 2025)